Experiment No. – 5							
Date of Performance:	**1st Aug 2026**						
Date of Submission:	**1st Aug 2026**						
Conceptual Understanding (02)	**Execution (1.5)**	**Problem Solving & Debugging(02)**	**Professional Conduct & Lab Ethics(02)**	**Output/ Result Accuracy (1.5)**	**Documentation & Result Analysis (01)**	**Experiment Total (10)**	**Sign with Date**

Experiment No. 5
Implement String operations

5.1 Aim: Implement String operations : Write a java program to perform following operations on strings, charAt(int index), length(), contains(), equals(), isEmpty(), concat(), replace(), split(),indexOf(), toLowerCase(), toUpperCase(), trim() and substring() .

5.2 Course Outcome: Apply fundamental Java programming concepts, including data types, operators, control structures, and functions, to develop basic programs.

5.3 Learning Objectives: To study the Strings in Java and various string functions.

5.4 Requirement: Java development kit (JDK as per OS)

5.5 Related Theory:

In Java, strings are objects that represent sequences of characters. They are widely used in Java programming for storing and manipulating textual data.

String Class: In Java, strings are instances of the String class, which is part of the java.lang package. This class provides numerous methods for manipulating strings, such as concatenation, substring extraction, searching, replacing, and more.

Immutable: Strings in Java are immutable, meaning their values cannot be changed after they are created. When you manipulate a string (e.g., by concatenating it with another string), it creates a new string object rather than modifying the original one. This immutability ensures thread safety and simplifies memory management.

String Literal: Creating a string using string literals, which are sequences of characters enclosed within double quotes

String Object: Creating strings using the new keyword and the String constructor.
String Pool: Java maintains a special memory area called the "string pool" to store string literals. When we create a string using a string literal, Java first checks the string pool. If a string with the same value already exists, it returns a reference to that string; otherwise, it creates a new string object in the pool. This improves memory efficiency by reducing duplicate string objects.

String Concatenation: Concatenate strings using the + operator or the concat() method.

Table 5.1 : String class methods

String class Method	Description
charAt()	Returns the character at the specified index (position)
codePointAt()	Returns the Unicode of the character at the specified index
codePointBefore()	Returns the Unicode of the character before the specified index
codePointCount()	Returns the number of Unicode values found in a string.
compareTo()	Compares two strings lexicographically
compareToIgnoreCase()	Compares two strings lexicographically, ignoring case differences
concat()	Appends a string to the end of another string
contains()	Checks whether a string contains a sequence of characters
contentEquals()	Checks whether a string contains the exact same sequence of characters of the specified CharSequence or StringBuffer
copyValueOf()	Returns a String that represents the characters of the character array
endsWith()	Checks whether a string ends with the specified character(s)
equals()	Compare two strings. Returns true if the strings are equal, and false if not
equalsIgnoreCase()	Compares two strings, ignoring case considerations

format()	Returns a formatted string using the specified locale, format string, and arguments
getBytes()	Converts a string into an array of bytes
getCharts()	Copies characters from a string to an array of chars
hashCode()	Returns the hash code of a string
indexOf()	Returns the position of the first found occurrence of specified characters in a string
intern()	Returns the canonical representation for the string object
isEmpty()	Checks whether a string is empty or not
join()	Joins one or more strings with a specified separator
lastIndexOf()	Returns the position of the last found occurrence of specified characters in a string
length()	Returns the length of a specified string
matches()	Searches a string for a match against a regular expression, and returns the matches
offsetByCodePoints()	Returns the index within this String that is offset from the given index by codePointOffset code points
regionMatches()	Tests if two string regions are equal
replace()	Searches a string for a specified value, and returns a new string where the specified values are replaced
replaceAll()	Replaces each substring of this string that matches the given regular expression with the given replacement
replaceFirst()	Replaces the first occurrence of a substring that matches the given regular expression with the given replacement
split()	Splits a string into an array of substrings
startsWith() startsWith()	Checks whether a string starts with specified characters
subSequence()	Returns a new character sequence that is a subsequence of this sequence
substring()	Returns a new string which is the substring of a specified string

toCharArray()	Converts this string to a new character array
toLowerCase()	Converts a string to lower case letters
toString()	Returns the value of a String object
toUpperCase()	Converts a string to upper case letters
trim()	Removes whitespace from both ends of a string
valueOf()	Returns the string representation of the specified value

Examples of String Methods:

charAt(): The Java String class charAt() method returns a char value at the given index number. The index number starts from 0 and goes to n-1, where n is the length of the string. It returns a StringIndexOutOfBoundsException, if the given index number is greater than or equal to this string length or a negative number.

Syntax: public char charAt(int index)

The method accepts index as a parameter. The starting index is 0. It returns a character at a specific index position in a string. It throws StringIndexOutOfBoundsException if the index is a negative value or greater than this string length.

Java String charAt() Method Examples

```
public class demo
{
        public static void main(String args[]){
        String name="SAKEC";
        char ch=name.charAt(4);//returns the char value at the 4th index
        System.out.println(ch);
        }
}
```

compareTo(): The Java String class compareTo() method compares the given string with the current string lexicographically. It returns a positive number, negative number, or 0. It compares strings on the basis of the Unicode value of each character in the strings.

If the first string is lexicographically greater than the second string, it returns a positive number (difference of character value). If the first string is less than the second string lexicographically, it returns a negative number, and if the first string is lexicographically equal to the second string, it returns 0.

if s1 > s2, it returns positive number

if s1 < s2, it returns negative number

if s1 == s2, it returns 0

Syntax: public int = String1.compareTo(String2)

The method accepts a parameter of type String that is to be compared with the current string.

Java String compareTo() Method Example:

Example:

```
public class demo1
{
	public static void main(String args[]){
	String s1="geeta";
	String s2="neeta";
	String s3="seeta";
	String s4="geeta";

	System.out.println(s1.compareTo(s2));
	System.out.println(s1.compareTo(s3));
	System.out.println(s1.compareTo(s4));

	}
}
```

concat():

The Java String class concat() method combines a specified string at the end of a string. It returns a combined string.

Syntax:

String String1 = String1.concat(String2)

Example:

```
public class Demo3
{
	public static void main(String args[])
	{
		String s1="Welcome ";
		s1=s1.concat("in SAKEC");
		System.out.println(s1);
```

```
        }
}
```

length():

The Java String class length() method finds the length of a string. The length of the Java string is the same as the Unicode code units of the string.

Syntax:

public int S= String.length()

Java String length() method example

```
public class LengthExample
{
        public static void main(String args[]){
        String s1="SAKEC";
        System.out.println("string length is: "+s1.length());
        }
}
```

5.6 Procedure:

Create a class, define 2 strings then apply all string functions one by one and observe and note the output after applying each string function.

5.7 Program and Output:

```
public class Main {
   public static void main(String[] args) {

      String s1 = " Hello Java ";
      String s2 = "Java";

      System.out.println("Original String: " + s1);

      // charAt()
      System.out.println("charAt(2): " + s1.charAt(2));

      // length()
      System.out.println("Length: " + s1.length());

      // contains()
      System.out.println("Contains 'Java': " + s1.contains("Java"));

      // equals()
      System.out.println("Equals s2: " + s1.equals(s2));

      // isEmpty()
      System.out.println("Is Empty: " + s1.isEmpty());
```

```
        // concat()
        System.out.println("Concat: " + s1.concat(" Programming"));

        // replace()
        System.out.println("Replace: " + s1.replace("Java", "World"));

        // split()
        String str = s1.trim();
        String[] words = str.split(" ");
        System.out.println("Split:");
        for (String w : words) {
            System.out.println(w);
        }

        // indexOf()
        System.out.println("Index of Java: " + s1.indexOf("Java"));

        // toLowerCase()
        System.out.println("Lower Case: " + s1.toLowerCase());

        // toUpperCase()
        System.out.println("Upper Case: " + s1.toUpperCase());

        // trim()
        System.out.println("Trim: " + s1.trim());

        // substring()
        System.out.println("Substring(6): " + s1.substring(6));
    }
}
```

Output:

```
Original String:  Hello Java
charAt(2): e
Length: 12
Contains 'Java': true
Equals s2: false
Is Empty: false
Concat:  Hello Java Programming
Replace:  Hello World
Split:
Hello
Java
Index of Java: 7
Lower Case:  hello java
Upper Case:  HELLO JAVA
Trim: Hello Java
Substring(6): Java
```

5.8 Conclusion:

This experiment helped us understand the use of the Java String class and its built-in methods. We performed operations such as charAt(), length(), contains(), concat(), replace(), split(), substring(), and others. We also observed that strings in Java are immutable, meaning every modification creates a new string object. These methods make string manipulation simple and efficient in Java applications.

5.9 Questions:

1. What does it mean for strings to be immutable?

Ans:Strings in Java are immutable, which means their contents cannot be changed after creation. Any modification creates a new String object

2. How do you create a string in Java?

Ans:Using a string literal:

String s = "Hello";

- Using the new keyword:

String s = new String("Hello");

3. What is the difference between using string literals and the new keyword to create strings?

Ans:

String Literal	new Keyword
Stored in the String Pool.	Creates a new object in heap memory.
Memory efficient because identical literals are shared.	Always creates a new object even if the same value exists.
Faster.	Uses more memory.

This version is suitable for a **Java practical/lab manual** and follows the requirements of the experiment.